

DSA 107

Data Science and Analytics Laboratory

Lab 1 report

Environment and first notebook

Andrei Tiniakov

Student ID: 26022121102

30 September 2026

Objective

The lab is set out to work with a famous penguin data set that has info about different penguins species. I had to use different libraries used by data scientists like pandas, numpy, and matplotlib.

Pandas allows us to work with data sets easily by representing data as a table. It helps us to load, inspect, and organize in tables. Its main table object is a **DataFrame** (a object that that is table like that we can work with), with rows and named columns.

Numpy helps us calculate data efficiently. In particular arrays (i.e. we could multiple an entire array by a factor really fast since numpy uses C language under the hood which is much faster then python)

Matplotlib lets us draw graphs and create figures to visualize data for easier understanding. Then we do multiple tasks related to the data set using those libraries, like counting rows, filtering data, and visualizing it with graphs.

Data

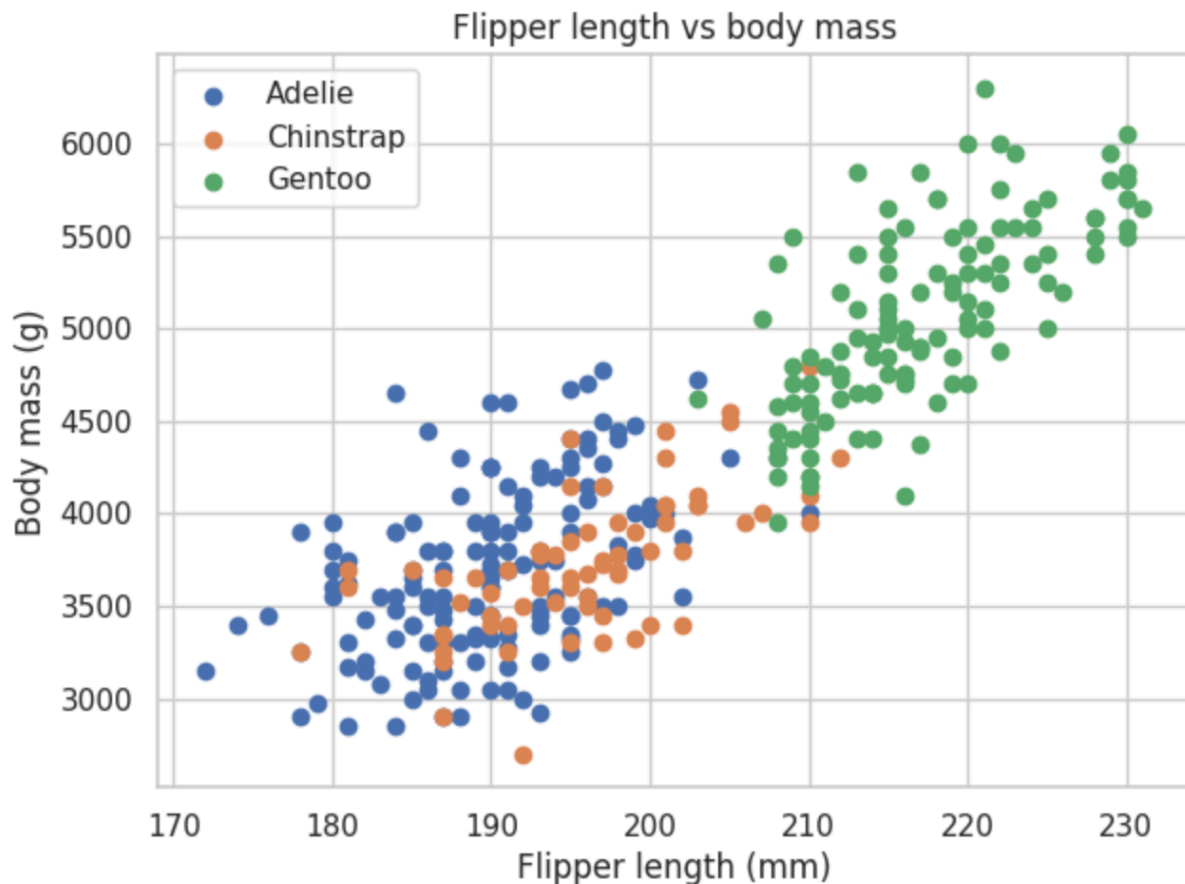
We used the Palmer Penguins dataset. We fetched it using a url and created a data frame using pandas. Since the source uses csv format we used the `pd.read_csv` function, but we wanted to load a json or another format we could call other functions provided by pandas

Method

We had a few tasks related to the data set, like:

- 1) getting first k entries (using `.head(k)`).
- 2) reading the shape of the data using `(.shape)` - here we get the count of rows and columns in the table.
- 3) getting info using `(.info)` - this helps us understand what columns we have and how those are typed (object, float, etc), and how much memory the data frame takes.
- 4) Getting some metrics for data using `describe()` - it gives us different statistics for each numeric column, like: count, mean (avg), std (standard deviation - how spread values are around the mean), percentiles (25%, 50%, 75%) as well as min and max.
- 5) Then we investigated missing data. Here we used `.isna()` (which stands for “is this value NA (Not Available)?”). We also used a sum function on top of isn result to get a summary for column by counting each True as 1 per column (this way we can know how many values are missing in each column to understand data completeness). Also calling `.sum(axis=1)` helps us count per row and not per column (be default it uses `axis=0`). This way we can count how many nulls each row has. I then realized that it was not what the question was about - task asked us to count rows with `axis=0`.
- 6) Then we learnt how to filter data or create one data set from another. For this we use this syntax - `penguins[["species", "another_field_name"]]` - this helps us get only k (2) columns from the data set. And then we can filter this data and create another one from it (`gentoo = penguins[penguins["species"] == "Gentoo"]`).
- 7) And finally we created a graph to visualize the data. For this we used matplotlib. `set_title`, `set_(x/y)label`, and `legend` helps us explain what graphs shows, what each axis means and what each color on the graph represents. And then `scatter` helps us to draw each entry as a dot

on the graph. And then at the end `.show()` renders the graph
Here is the result:



Result

We managed to get counts of columns, rows, nullable rows, and visualize the data for better comprehension. The dataset contained 344 penguins and 7 columns. The sex column had the most missing values, with 11 missing entries. My separate exploration showed that two rows had five missing values each.

Discussion

The data shows us the difference in different penguins species as well as general data set information.

Reflection

The difficult part was just learning the library API (functions, fields, etc). I believe that next time I would just spend less time checking what each function does. Also the process of commenting the code to explain what each function is for was helpful to better understand it once I came back to read the code again.

The feedback showed that I had answered Q2 about rows, although the question asked about columns. My extra exploration was useful, but I should have checked that my written answer

matched the question. Next time, I will reread each question before answering and check that the output supports my answer.

I also learned that `info()` prints its own summary and returns `None`, which explains the extra `None` in my output. Another lesson was that Colab does not automatically display every expression in a cell. Next time, I will check that both the table preview and the count are visible.